

Spell Arena

Real-Time Multiplayer Gesture-Based Combat

Final Presentation

Joe Ampfer

ASE 485

April 26, 2026

The Problem

Two intertwined problems I set out to solve:

1. Multiplayer at zero server cost

- Building competitive networked combat without paying for dedicated servers.

2. A VR-like experience without a VR headset

- Games like *War of Wizards* require a \$300+ headset.
- Many players want gesture-based spell combat but can't (or won't) pay that.

The Solution

Problem	Solution
Multiplayer with no server bill	Unity Netcode for GameObjects + Facepunch.Steamworks + P2P host-client topology. Steam handles NAT, relay, lobbies, matchmaking.
VR-like input	Google MediaPipe gesture recognition over a standard webcam (built last semester, kept and extended).

Tech stack: Unity, Netcode for GameObjects, Facepunch.Steamworks, Google MediaPipe ML.

Demo

Video: <https://youtu.be/3Cgrsk3zq-w>

What to watch for:

- Steam friend invite into a lobby
- Gesture-cast spells from a webcam
- Networked projectiles + synced damage
- Win screen, rematch flow, and disconnect handling

What I Learned

- **Multiplayer fundamentals** — server-authoritative model, RPCs, NetworkVariables.
- **P2P host-client architecture** and its trade-offs vs. dedicated servers.
- **Unity Netcode for GameObjects** — spawn / despawn lifecycle, transform sync, ownership.
- **Network delay handling** — action anticipation and deterministic projectiles.
- **Steam lobbies & matchmaking** through Facepunch.Steamworks.
- **Post-processing** in Unity — bloom, color grading, chromatic aberration.

What I Discovered

Three real shifts in how I think about building software:

- **State machines for UI panels.** Coming from React I conditionally rendered everything; for 5 lobby panels reacting to NetworkManager callbacks, a finite state machine is dramatically cleaner.
- **Post-processing is a force multiplier.** I didn't know it existed in Unity until I saw an NKU game jam entry. One tutorial later, the arena looked dramatically better.
- **C# events make the observer pattern concrete.** The pattern was abstract to me before; in C# it's just `event` + `+=`, and decoupling subsystems suddenly felt natural.

Issues I Faced

1. **Network latency** made the game feel unfair to remote clients.
2. **Lobby scene chaos** — 5 panels all reacting to `NetworkManager.Singleton` callbacks (`OnClientConnected/Disconnected`, lobby join/leave/change) in one giant file.
3. **Race condition on match start** — game began before clients finished loading the scene.
4. **Low FPS on laptops** during testing.
5. **Choppy host camera** when moving the player via `Transform`.

How I Solved Them

Issue	Solution
Network latency	Client-side action anticipation + deterministic projectiles (host and clients run identical sims from the same start state)
Lobby panel chaos	Refactored into a UI state machine — each panel is a state, callbacks fire transitions, side effects centralized
Scene-load race	Host now waits on a per-client "scene loaded" callback before starting the match

Issue	Solution
Low FPS on laptops	Settings menu with adjustable render scale
Choppy host camera	Detached camera from player Transform and added smoothing to camera updates

How AI Helped

Tools I used: ChatGPT first, then **Claude (web)** for better answers; **Cursor chat** for in-context code questions.

- **Win:** AI taught me latency compensation, action anticipation, and deterministic projectiles — the foundation for fixing the unfairness problem. Deeper write-up: <https://github.com/joseph-ampfer/LearningWithAI/blob/main/presentation/3-30-Presentation.pdf>
- **Win:** When the lobby was a mess, asking Claude about state machines turned tangled callbacks into clean, testable states.

- **Cautionary tale:** ChatGPT once told me to delay `StartHost()` / `StartClient()` until both players readied up. That broke Steam's "Invite Friend" overlay (which needs the lobby created first) and made scene loads painfully slow. I rejected the suggestion. **Lesson: AI is a collaborator, not an authority.**

Progress Since S1P

Week	Accomplishment
Week 1	Action anticipation for high-latency clients on spell cast
Week 2	Fixed projectile jitter via client-side duplication
Week 3	Networked all 17 spells (full spellbook)
Week 4	Full lobby system, spell cooldown UI, multiplayer pause menu
Week 5	Disconnect messages, decorated arena, post-processing, settings menu, final testing, demo video

Publicly Published

Resource
GitHub repo
Project website (Hugo) + Windows download
Demo video
Canvas page

Learning with AI — Two Topics

- Topic 1: Networking Fundamentals for Multiplayer Games
 - Server authority, RPCs (`ServerRpc` / `ClientRpc`), latency compensation.
 - Marp PDF in [LearningWithAI](#) repo.
- Topic 2: Unity Netcode for GameObjects
 - `NetworkObject` spawn/despawn, `NetworkVariable<T>` , refactoring single-player to multiplayer.
 - Marp PDF in [LearningWithAI](#) repo.
- Bonus AI-learning artifact: [3-30-Presentation.pdf](#) —

Thank You

Questions?

- GitHub: https://github.com/joseph-ampfer/ninja_wizards_hands
- Demo: <https://youtu.be/3Cgrsk3zq-w>

Joe Ampfer — ASE 485